0.31. Deux lettres à la fois

Le 0.30, plus une deuxième lettre : deux un_pas dans la même boucle, et les deux bougent ensemble.

1. Ce qu'il faut comprendre

C'est le 0.30, plus une lettre, le B, qui descend pendant que le A avance.

Ce qui est nouveau ici : deux appels à `un_pas` dans la même boucle, l'un après l'autre. Comme `un_pas` n'attend pas, les deux se font **dans la même image** : les deux lettres bougent **ensemble**. Avec `deplace`, le B aurait attendu la fin du trajet du A.

Chaque lettre a ses deux variables : `xa`, `ya` pour le A ; `xb`, `yb` pour le B.

Le B s'arrête en ligne 16 : le `if (yb < 16)` ne le laisse plus avancer après, pour qu'il ne touche pas les nombres de la ligne 17.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <poser>` → 0.1.3. La fonction poser() — poser une tuile
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
> - `#include <un_pas>` → 0.29.4. La fonction un_pas() — un seul pas, sans attendre
> - `#include <nombre>` → 0.8.3. La fonction nombre() — écrire un nombre

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <poser>  // pose une tuile sur une case du fond`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de

	l'employer. (Le commentaire le dit : « pose une tuile sur une case du fond ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include <un_pas>  // fait faire un seul pas à une tuile`	Demande à la console « un_pas » (ce que c'est : fait faire un seul pas à une tuile). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « fait faire un seul pas à une tuile ».)
`#include <nombre>  // écrit un nombre en chiffres`	Demande à la console « nombre » (ce que c'est : écrit un nombre en chiffres). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « écrit un nombre en chiffres ».)
`uint8_t xa = 0;  // le A : sa colonne…`	Une variable : une boîte appelée xa, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « le A : sa colonne… ».)
`uint8_t ya = 0;  //  …et sa ligne. Il file vers la DROITE.`	Une variable : une boîte appelée ya, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « …et sa ligne. Il file vers la DROITE. ».)
`uint8_t xb = 0;  // le B : sa colonne…`	Une variable : une boîte appelée xb, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « le B : sa colonne… ».)
`uint8_t yb = 2;  //  …et sa ligne. Il file vers le BAS.`	Une variable : une boîte appelée yb, qui retient un nombre de 0 à 255, et qui commence à 2. (Le commentaire le dit : « …et sa ligne. Il file vers le BAS. ».)
`uint8_t images = 0;  // le chronomètre, en images`	Une variable : une boîte appelée images, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « le chronomètre, en images ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`poser(xa, ya, ALPHABET[0]);  // les deux lettres à leur place de départ`	Appel de la fonction poser de la console, avec les arguments xa, ya, ALPHABET[0]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « les deux lettres à leur place de départ ».)
`poser(xb, yb, ALPHABET[1]);  // ALPHABET[1] : la 2e lettre, B`	Appel de la fonction poser de la console, avec les arguments xb, yb, ALPHABET[1]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « ALPHABET[1] : la 2e lettre, B ».)
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`images++;`	images augmente de 1.
`if (images == 15) {  // 4 fois par seconde :`	Une décision : si images est égal à 15, on fait ce qui est entre les accolades. Sinon, on passe. (Le commentaire le dit : « 4 fois par seconde : ».)

`images = 0;`	images prend une nouvelle valeur : 0.
`un_pas(xa, ya, ALPHABET[0], 1, 0);` `// le A : un pas à droite`	Appel de la fonction un_pas de la console, avec les arguments xa, ya, ALPHABET[0], 1, 0. Ce qu'elle fait : UN pas, tout de suite, sans attendre : 1 vers la droite ou le bas, -1 vers la gauche ou le haut, 0 immobile. Ne bloque pas : à appeler dans la boucle, pour faire bouger plusieurs choses à la fois. un_pas(x, y, ALPHABET[0], 1, 0); range la nouvelle place dans x et y. (Le commentaire le dit : « le A : un pas à droite ».)
`// La ligne ajoutée : le B, un pas en bas — DANS LA MÊME IMAGE que le A,`	Un commentaire : une phrase pour celui qui lit (« La ligne ajoutée : le B, un pas en bas — DANS LA MÊME IMAGE que le A, »). La console l'ignore.
`// puisque un_pas n'attend pas. Tant que yb < 16 seulement.`	Un commentaire : une phrase pour celui qui lit (« puisque un_pas n'attend pas. Tant que yb < 16 seulement. »). La console l'ignore.
`if (yb < 16) un_pas(xb, yb, ALPHABET[1], 0, 1);`	Une décision : si yb est plus petit que 16) un_pas(xb, yb, ALPHABET[1], 0, 1 alors on fait « ». Sinon, on passe.
`nombre(12, 17, xa);  // la colonne du A, pour la voir changer`	Appel de la fonction nombre de la console, avec les arguments 12, 17, xa. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand. (Le commentaire le dit : « la colonne du A, pour la voir changer ».)
`nombre(16, 17, yb);  // la ligne du B`	Appel de la fonction nombre de la console, avec les arguments 16, 17, yb. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand. (Le commentaire le dit : « la ligne du B ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1175 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        72 o  ← la console, à l'allumage  — la mise en route de la console
  un_pas()        117 o  ← main()  — une fonction fournie par le langage
  main()          131 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases, EcrireNombre  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← effacer(), poser()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  EcrireNombre     33 o  ← nombre()  — écrit un nombre en chiffres
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
```

```
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert,
Notes
Fonctions du langage appelées :
  poser()   × 3    39 o sur place, dans un_pas(), main()  + AdresseCase (10 o)
  nombre()  × 2    20 o sur place, dans main()  + EcrireNombre (33 o)
  effacer() × 1    15 o sur place, dans un_pas()  + AdresseCase (10 o), EffacerCases (12 o)
  image()   × 1     3 o sur place, dans main()  + AttendreImage (28 o)
Tes fonctions appelées :
  un_pas()  × 2    44 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <un_pas>     // fait faire un seul pas à une tuile
#include <nombre>     // écrit un nombre en chiffres
```

2. UNE VARIABLE — XA

```
uint8_t xa = 0;       // le A : sa colonne…
```

3. UNE VARIABLE — YA

```
uint8_t ya = 0;       //       …et sa ligne. Il file vers la DROITE.
```

4. UNE VARIABLE — XB

```
uint8_t xb = 0;       // le B : sa colonne…
```

5. UNE VARIABLE — YB

```
uint8_t yb = 2;       //       …et sa ligne. Il file vers le BAS.
```

6. UNE VARIABLE — IMAGES

```
uint8_t images = 0;   // le chronomètre, en images
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

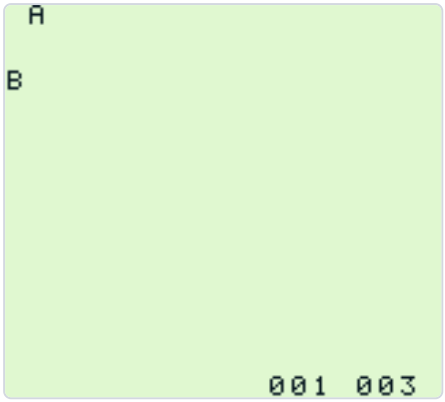
```
int main() {
  poser(xa, ya, ALPHABET[0]);   // les deux lettres à leur place de départ
  poser(xb, yb, ALPHABET[1]);   // ALPHABET[1] : la 2e lettre, B

  while (true) {
    image();
    images++;
    if (images == 15) {         // 4 fois par seconde :
      images = 0;
      un_pas(xa, ya, ALPHABET[0], 1, 0);               // le A : un pas à droite
      // La ligne ajoutée : le B, un pas en bas — DANS LA MÊME IMAGE que le A,
      // puisque un_pas n'attend pas. Tant que yb < 16 seulement.
      if (yb < 16) un_pas(xb, yb, ALPHABET[1], 0, 1);
      nombre(12, 17, xa);       // la colonne du A, pour la voir changer
      nombre(16, 17, yb);       // la ligne du B
    }
  }
}
```

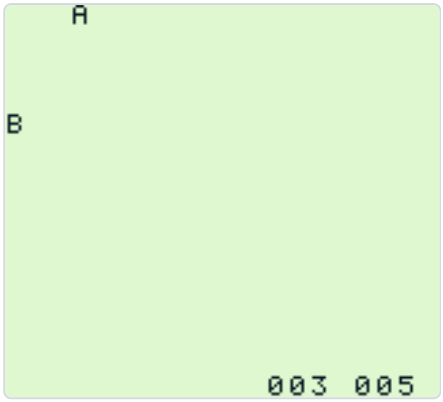
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

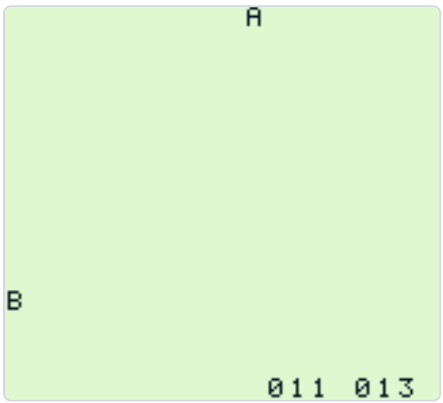
la toute première image — la console vient de démarrer — image 1



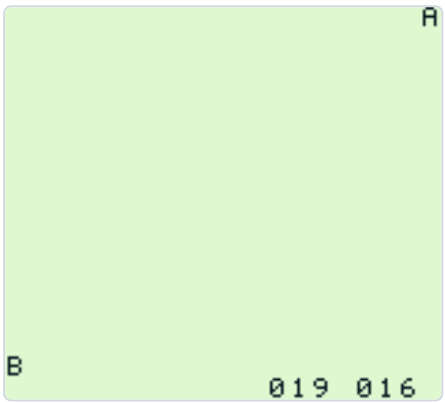
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> Un A qui file vers la droite sur la ligne du haut, et un B qui descend en même temps sur la gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A et le B avancent (4 et 4)
- ✓ du même nombre de pas
- ✓ le A s'arrête au bord : colonne 19 (xa = 19)
- ✓ le B s'arrête en ligne 16 (yb = 16)

Ensuite : **0.31.1. Deux lettres à la fois — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++